

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

ДОПУСТИТЬ К ЗАЩИТЕ

Заведующий кафедрой № 43

должность, уч. степень, звание

подпись, дата

М.Ю. Охтилев
инициалы, фамилия

БАКАЛАВРСКАЯ РАБОТА

на тему

Разработка real-time интерфейса с поддержкой горизонтального масштабирования
WebSocket-соединений для системы отслеживания задач

выполнена	Зайцевым Александром Сергеевичем	
	фамилия, имя, отчество студента в творительном падеже	
по направлению подготовки	09.03.04	Программная инженерия
	код	наименование направления
направленности	02	Проектирование программных систем
		систем

Студент группы № 4131з

подпись, дата

А.С. Зайцев
инициалы, фамилия

Руководитель
канд. техн. наук, доцент

подпись, дата

А.В. Фомин
инициалы, фамилия

Санкт-Петербург 2026

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

федеральное государственное автономное образовательное учреждение высшего
образования

«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

УТВЕРЖДАЮ

Заведующий кафедрой №43 _____ М.Ю. Охтилев

ЗАДАНИЕ НА ВЫПОЛНЕНИЕ БАКАЛАВРСКОЙ РАБОТЫ

студенту группы 4131з Зайцев Александр Сергеевич

на тему: Разработка real-time интерфейса с поддержкой горизонтального
масштабирования WebSocket-соединений для системы отслеживания задач
утвержденную приказом ГУАП от «___» _____ 2026 г. № _____

Цель работы: разработать и исследовать real-time интерфейс для системы
отслеживания задач, в котором WebSocket-соединения обслуживаются
несколькими экземплярами серверного приложения, а события об изменении
сущностей корректно доставляются клиентам независимо от узла
подключения.

Задачи, подлежащие решению: проанализировать предметную область и
подходы к масштабированию WebSocket-соединений; исследовать исходную
реализацию real-time контура; спроектировать распределенную архитектуру с
межузловой доставкой событий; реализовать решение в отдельной ветке
проекта; подготовить стенд и провести испытания.

Содержание работы (основные разделы): анализ предметной области,
постановка задачи, проектирование масштабируемого real-time контура,
реализация, испытания и оценка результатов.

Срок сдачи работы «___» _____ 2026 г.

Руководитель

канд. техн. наук, доцент

Фомин Александр Владимирович

должность, уч. степень, звание

подпись, дата

фамилия, имя, отчество

Задание принял к исполнению студент группы № 4131з

А.С. Зайцев

подпись, дата

инициалы, фамилия

РЕФЕРАТ

Выпускная квалификационная работа содержит 52 страницы, 6 рисунков, 2 таблицы, список использованных источников из 20 наименований и 3 приложения.

Ключевые слова: real-time интерфейс, WebSocket, SignalR, горизонтальное масштабирование, Redis backplane, система отслеживания задач, межузловая доставка событий.

Актуальность работы обусловлена необходимостью обеспечения корректного real-time взаимодействия пользователей системы отслеживания задач при переходе от одного экземпляра серверного приложения к многосерверной конфигурации.

Цель работы — разработать и исследовать real-time интерфейс для системы отслеживания задач, в котором WebSocket-соединения обслуживаются несколькими экземплярами серверного приложения, а события об изменении сущностей корректно доставляются клиентам независимо от узла подключения.

Полученные результаты: разработан распределенный real-time контур на базе SignalR и Redis backplane, подготовлен воспроизводимый стенд с несколькими экземплярами, реализованы диагностические средства и проведена экспериментальная проверка межузловой доставки событий.

СОДЕРЖАНИЕ

РЕФЕРАТ	1
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	3
ВВЕДЕНИЕ	4
1. Анализ предметной области и существующих подходов	8
1.1 Real-time взаимодействие в системах отслеживания задач	8
1.2 WebSocket и SignalR как основа real-time контура	9
1.3 Ограничение одноузловой WebSocket-архитектуры	9
1.4 Сравнение подходов к масштабированию	10
1.5 Вывод по главе	12
2. Анализ целевой системы и постановка задачи	13
2.1 Характеристика целевой системы	13
2.2 Исходная реализация real-time взаимодействия	14
2.3 Требования к разрабатываемому решению	15
2.4 Выбор платформы и инструментальных средств	16
2.5 Постановка задачи	16
3. Проектирование масштабируемого real-time контура	18
3.1 Исходная архитектура	18
3.2 Целевая архитектура	19
3.3 Поток real-time события	20
3.4 Модель групп и повторной подписки	21
3.5 Диагностический контур	22
3.6 Семантика доставки и ограничения	22
4. Реализация решения в целевой системе	24
4.1 Организация работ	24
4.2 Изменения серверной части	24
4.3 Изменения клиентской части	26
4.4 Проверочный клиент	27
4.5 Инфраструктурные изменения	27
4.6 Автоматизированный сценарий проверки	28
5. Испытания и оценка результатов	30
5.1 Цель и программа испытаний	30
5.2 Стенд испытаний	30
5.3 Результат без Redis backplane	31
5.4 Результат с Redis backplane	31
5.5 Серийная проверка и задержка доставки	32
5.6 Проверка восстановления соединения и отказа узла	33
5.7 Сравнительная таблица результатов	33
5.8 Проверка входной точки стенда	34
5.9 Оценка достоверности результатов	34
5.10 Вывод по испытаниям	35
ЗАКЛЮЧЕНИЕ	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	37
ПРИЛОЖЕНИЕ А	38
ПРИЛОЖЕНИЕ Б	39
ПРИЛОЖЕНИЕ В	43

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

Сокращение	Расшифровка
API	Application Programming Interface, программный интерфейс приложения
ASP.NET Core	платформа для разработки веб-приложений на языке C#
HTTP	HyperText Transfer Protocol, протокол передачи гипертекста
JSON	JavaScript Object Notation, текстовый формат обмена данными
nginx	обратный прокси-сервер и балансировщик нагрузки
Redis	система хранения данных в памяти, используемая как backplane
SignalR	библиотека ASP.NET Core для real-time взаимодействия
UI	User Interface, пользовательский интерфейс
URL	Uniform Resource Locator, адрес ресурса
WebSocket	сетевой протокол двусторонней связи поверх одного TCP-соединения
ВКР	выпускная квалификационная работа

ВВЕДЕНИЕ

Современные системы отслеживания задач используются не только как хранилище карточек, статусов и комментариев, но и как рабочая среда для совместной координации действий команды. Пользователи ожидают, что изменения задач, комментариев, вложений, статусов и участников будут отображаться без ручного обновления страницы. Поэтому real-time интерфейс в такой системе следует рассматривать не как декоративный элемент, а как часть прикладного контура совместной работы, обеспечивающую оперативную синхронизацию состояния между участниками [15], [19].

Актуальность такого контура усиливается ростом сложности современных web-продуктов. Интерфейсы всё чаще объединяют несколько связанных сущностей, фоновые операции, уведомления, совместное редактирование и автоматизированные подсказки. На фоне распространения интеллектуальных ассистентов и AI-инструментов пользовательские ожидания также смещаются от статичных CRUD-экранов к событийным рабочим пространствам: система может не только ждать ручного действия пользователя, но и автоматически формировать комментарий, обновлять статус, предлагать исполнителя или дополнять описание задачи. В рамках данной ВКР не утверждается, что такая AI-функциональность уже реализована в целевом продукте; она рассматривается как отраслевой контекст, показывающий, почему способность интерфейса быстро и согласованно распространять изменения становится важной инженерной характеристикой.

Одним из распространенных способов реализации такого взаимодействия является использование WebSocket-соединений и серверного механизма отправки событий клиенту. В веб-приложениях на платформе ASP.NET Core для этого часто применяется SignalR, предоставляющий модель хабов, клиентских событий, групп и автоматического выбора транспорта [1], [2]. Однако простая реализация real-time взаимодействия на

одном экземпляре серверного приложения имеет принципиальное ограничение: сведения о подключениях, группах и активных клиентах находятся в памяти конкретного процесса. При запуске нескольких экземпляров серверного сервиса без межузловой синхронизации каждый экземпляр знает только о своих соединениях. В результате пользователь, подключенный к одному узлу, может не получить событие, созданное на другом узле [3].

Актуальность работы определяется необходимостью перехода от одноузлового real-time решения к архитектуре, допускающей горизонтальное масштабирование WebSocket-соединений. Для системы отслеживания задач это особенно важно, поскольку рост числа пользователей, длительных соединений и фоновых событий не должен разрушать целостность пространства оперативной синхронизации. REST API остается основным механизмом выполнения команд и чтения состояния, однако сам по себе не решает задачу отправки изменений другим активным участникам. Real-time слой дополняет REST: HTTP-запрос фиксирует изменение состояния, а WebSocket/SignalR-событие распространяет это изменение пользователям, которые сейчас работают с соответствующей сущностью [3], [4].

Научно-техническая проблема работы состоит в обеспечении корректной доставки real-time событий в многосерверной конфигурации, где состояние WebSocket-подключений локально для отдельных серверных экземпляров. Решение этой проблемы требует выбора архитектуры межузловой доставки, реализации ее в целевой системе и экспериментального подтверждения того, что ограничение одноузловой WebSocket-архитектуры устранено.

Объектом исследования являются процессы real-time взаимодействия пользователей в веб-системах совместной работы над сущностями системы отслеживания задач.

Предметом исследования являются архитектурные и программные средства построения WebSocket-контура с горизонтальным

масштабированием, межузловой доставкой событий и восстановлением соединений в системе на базе SignalR.

Цель работы: разработать и исследовать real-time интерфейс для системы отслеживания задач, в котором WebSocket-соединения обслуживаются несколькими экземплярами серверного приложения, а события об изменении сущностей корректно доставляются клиентам независимо от узла подключения.

Для достижения цели поставлены следующие задачи:

1. Проанализировать предметную область real-time взаимодействия в системах совместной работы.
2. Рассмотреть ограничения одноузловой WebSocket-архитектуры при горизонтальном масштабировании.
3. Сравнить подходы к масштабированию real-time контура и обосновать выбор архитектуры.
4. Проанализировать текущую реализацию real-time взаимодействия в целевой системе.
5. Спроектировать масштабируемый real-time контур с несколькими экземплярами серверного сервиса.
6. Реализовать выбранное решение в целевой системе.
7. Подготовить минимальный воспроизводимый стенд для проверки результата.
8. Провести испытания в режимах с межузловой доставкой и без нее.
9. Оценить результаты и сформулировать вывод об устранении ограничения одноузловой архитектуры.

Элемент новизны работы заключается в инженерно-исследовательском обосновании и экспериментальной проверке распределенного real-time контура для системы отслеживания задач. В рамках работы не разрабатывается новый сетевой протокол, но выполняется авторская композиция архитектурного решения, включающая SignalR, несколько экземпляров серверного сервиса, балансировщик, Redis backplane,

диагностический контур, клиентское восстановление подписок и воспроизводимую программу испытаний [2], [4], [8], [10].

Практическая значимость работы состоит в создании прототипа масштабируемого real-time контура, который может быть использован как основа для дальнейшего развития системы отслеживания задач. Подготовленные конфигурации, диагностические средства и сценарии испытаний позволяют воспроизводимо демонстрировать переход от одноузловой реализации к режиму с несколькими экземплярами и корректной межузловой доставкой событий.

Структура пояснительной записки и оформление выполнены с учётом ГОСТ 7.32-2017 [11] и ГОСТ Р 7.0.100-2018 [12], а также методических требований кафедры к подготовке учебных отчетных работ [13].

1. Анализ предметной области и существующих подходов

1.1 Real-time взаимодействие в системах отслеживания задач

Системы отслеживания задач предназначены для фиксации, распределения и контроля выполнения работ. В таких системах ключевыми сущностями являются задачи, отчеты, комментарии, вложения, исполнители, статусы, команды и рабочие пространства. В отличие от статичных справочников, эти сущности часто изменяются несколькими пользователями в течение короткого времени. Один участник может изменить статус задачи, второй добавить комментарий, третий прикрепить файл или уточнить шаг воспроизведения ошибки [16], [18].

Если изменения становятся видимыми только после ручного обновления страницы, у пользователей возникает рассогласованное представление о состоянии системы. Это приводит к дублированию действий, работе с устаревшими данными и дополнительным коммуникационным издержкам. Поэтому для систем отслеживания задач важна поддержка согласованного представления о состоянии объекта: пользователь должен понимать, что объект уже изменился, кто присоединился к работе, какие комментарии или вложения появились и какие поля были обновлены [15], [19].

В контексте данной работы real-time интерфейс определяется как интерфейсный и серверный контур, обеспечивающий оперативное распространение событий изменения сущностей между клиентами. Такой контур включает клиентское WebSocket-соединение, серверный хаб, модель подписок, механизм групп, доставку событий и обработку переподключений [1], [2], [5], [6], [14].

1.2 WebSocket и SignalR как основа real-time контура

WebSocket обеспечивает двусторонний обмен данными поверх одного длительно открытого соединения. Для web-интерфейсов совместной работы

это удобнее, чем периодический polling, поскольку сервер может самостоятельно отправлять клиентам события сразу после изменения состояния. В платформе ASP.NET Core над WebSocket часто используется SignalR. Он предоставляет более высокоуровневую модель: серверные хабы, клиентские методы, группы, отправку событий конкретным пользователям или всем участникам группы [1], [2].

Для системы отслеживания задач особенно полезна модель групп. Клиент может вступить в группу, соответствующую конкретному отчету или задаче, после чего сервер отправляет события изменения только заинтересованным участникам. Такой подход снижает лишний трафик и делает real-time контур предметно ориентированным: событие связано не с абстрактным каналом, а с конкретной сущностью системы [5].

В то же время группы SignalR являются оперативным механизмом доставки, а не долговременным источником истины. Сведения о членстве соединений в группах хранятся в памяти серверного процесса и не должны использоваться как единственная база для авторизации, аудита или долговременного учета присутствия. Это обстоятельство важно при проектировании масштабируемого контура, поскольку при переподключении клиент должен заново восстановить необходимые подписки [5], [6].

1.3 Ограничение одноузловой WebSocket-архитектуры

Одноузловая архитектура означает, что все WebSocket-соединения обслуживаются одним экземпляром серверного сервиса. Такой вариант прост в реализации и достаточен для раннего прототипа. Сервер хранит локальную таблицу активных соединений и групп, а при изменении сущности отправляет событие клиентам, подключенным к этому же процессу [3], [5].

Проблема возникает при горизонтальном масштабировании. Если приложение запускается в нескольких экземплярах, входящий трафик распределяется между ними балансировщиком. Клиент А может оказаться подключенным к `app-api-1`, а клиент В — к `app-api-2`. Если событие

изменения отчета создано на первом узле, то без межузловой синхронизации второй узел не узнает, что нужно отправить событие своим клиентам.

Формально система уже имеет два серверных экземпляра, но пространство оперативной синхронизации фрагментируется на независимые области [3], [10].

Для системы совместной работы это является функциональным дефектом. Пользователь не обязан знать, через какой серверный процесс он подключен. Ожидаемое свойство интерфейса состоит в том, что изменение сущности доставляется всем заинтересованным участникам. Следовательно, поддержка горизонтального масштабирования WebSocket-соединений должна включать не только запуск нескольких процессов, но и межузловую доставку событий [3], [15].

1.4 Сравнение подходов к масштабированию

В рамках работы рассматривались несколько подходов к построению масштабируемого real-time контура [3], [4], [7], [9], [10]. Сравнение выполнялось по критериям, важным для целевой системы: решает ли подход межузловую доставку, сохраняет ли модель SignalR-групп, подходит ли для локально развернутого стенда, требует ли переработки доменных событий и может ли быть проверен в локальном Docker Compose.

Таблица 1 — Сравнение подходов к масштабированию WebSocket-соединений

N	Подход	Межузловая доставка	Применимость к целевой системе	Ограничения
1	Один экземпляр серверного сервиса	Нет	Подходит только для раннего прототипа	Нет горизонтального масштабирования
2	Несколько экземпляров без Redis backplane	Нет	Позволяет воспроизвести проблему	Локальные группы фрагментируются по узлам
3	Закрепление клиента за узлом	Нет, если нет отдельной синхронизации	Упрощает маршрутизацию одного клиента	Не доставляет события клиентам других узлов
4	Redis backplane для SignalR	Да	Сохраняет SignalR-группы, локальный стенд и локальную проверку	Зависит от Redis и сетевой задержки
5	Брокер сообщений или шина событий	Да	Подходит для событийно-ориентированной архитектуры	Требует переработки доменных событий
6	Управляемый real-time сервис	Да	Уместен при выносе real-time слоя во внешний сервис	Снижает контроль над локально развернутым контуром

Для реализации выбран подход 4 — Redis backplane для SignalR.

Подход 1 не решает задачу горизонтального масштабирования, поскольку все WebSocket-соединения остаются привязанными к одному процессу. Подход 2 воспроизводит проблему: каждый экземпляр хранит собственные локальные группы SignalR, поэтому событие, созданное на одном узле, не попадает к клиентам другого узла. Подход 3 улучшает предсказуемость маршрутизации отдельного клиента, но не решает межузловую доставку: если клиент А закреплен за `app-api-1`, а клиент В — за `app-api-2`, событие из памяти первого узла всё равно не становится известным второму.

Подход 5 с брокером сообщений или шиной событий подходит для более строгой событийно-ориентированной архитектуры, долговременных интеграций и повторной обработки событий. Однако для текущей задачи он избыточен: real-time события интерфейса являются короткоживущими событиями интерфейса, а источником истины остается PostgreSQL. Внедрение журнала событий потребовало бы отдельного проектирования схем событий, групп потребителей, паттернов outbox/inbox и правил повторной доставки. Подход 6 снижает объем локальной инфраструктуры, но уменьшает контроль над исследуемым контуром и усложняет воспроизводимую локальную демонстрацию.

Таким образом, Redis backplane для SignalR выбран не как произвольная инфраструктурная настройка, а как инженерный компромисс.

Он сохраняет существующий код отправки событий в SignalR-группы, требует минимальной переработки доменной логики, подходит для Docker Compose стенда и позволяет напрямую проверить свойство межузловой доставки [4], [9].

1.5 Вывод по главе

Анализ показал, что real-time интерфейс в системе отслеживания задач должен рассматриваться как контур совместной работы над сущностями, а не только как клиентский WebSocket. Главным ограничением одноузловой реализации является локальность сведений о соединениях и группах. Для устранения этого ограничения требуется архитектура, в которой несколько экземпляров серверного сервиса обмениваются real-time событиями через межузловую механизм. Для рассматриваемой системы рациональным решением является использование Redis backplane в составе SignalR [3], [4], [15].

2. Анализ целевой системы и постановка задачи

2.1 Характеристика целевой системы

Целевая система представляет собой веб-систему для работы с отчетами, задачами, комментариями, вложениями и связанными сущностями. В работе рассматривается не вся продуктовая инфраструктура, а минимальный контур real-time взаимодействия: прикладной серверный сервис `app-api`, PostgreSQL, Redis, обратный прокси-сервер `nginx` и отдельный проверочный клиент.

В качестве целевой системы рассматривается существующая кодовая база продукта, в которой уже реализованы серверный сервис, клиентское приложение и инфраструктурные конфигурации. Для исследования выделен контур, достаточный для проверки real-time взаимодействия: SignalR-хаб, механизм групп, обратный прокси-сервер, база данных, Redis и проверочный клиент.

На рисунке 1 показан дашборд целевой системы. Пользователь работает со списком отчетов команды, видит ответственных, участников и состояние отдельных задач. Такой экран показывает прикладной контекст real-time синхронизации: изменения в отчетах и связанных сущностях должны становиться видимыми участникам без ручного обновления страницы.

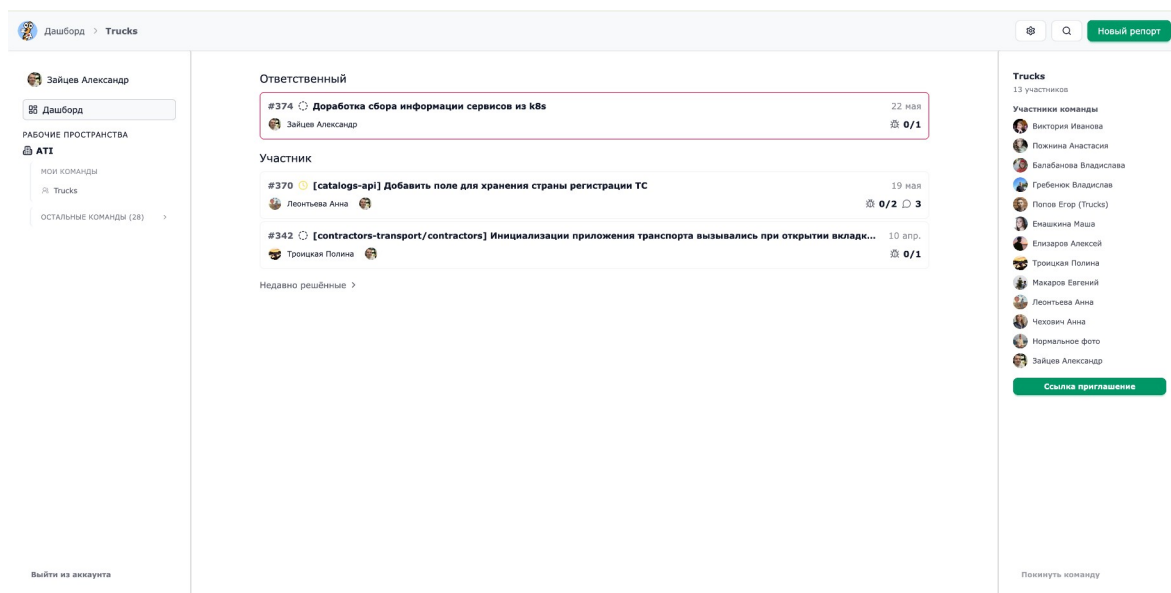


Рисунок 1 — Дашборд целевой системы со списком отчетов команды

На рисунке 2 показана страница отчета. На ней представлены заголовок отчета, статус, ответственный, участники, баги, фактический и ожидаемый результат, шаги воспроизведения и комментарии. Именно эти сущности формируют предметную область real-time контура: при изменении отчета, бага, комментария или статуса сервер должен распространить событие другим пользователям, работающим с тем же отчетом.

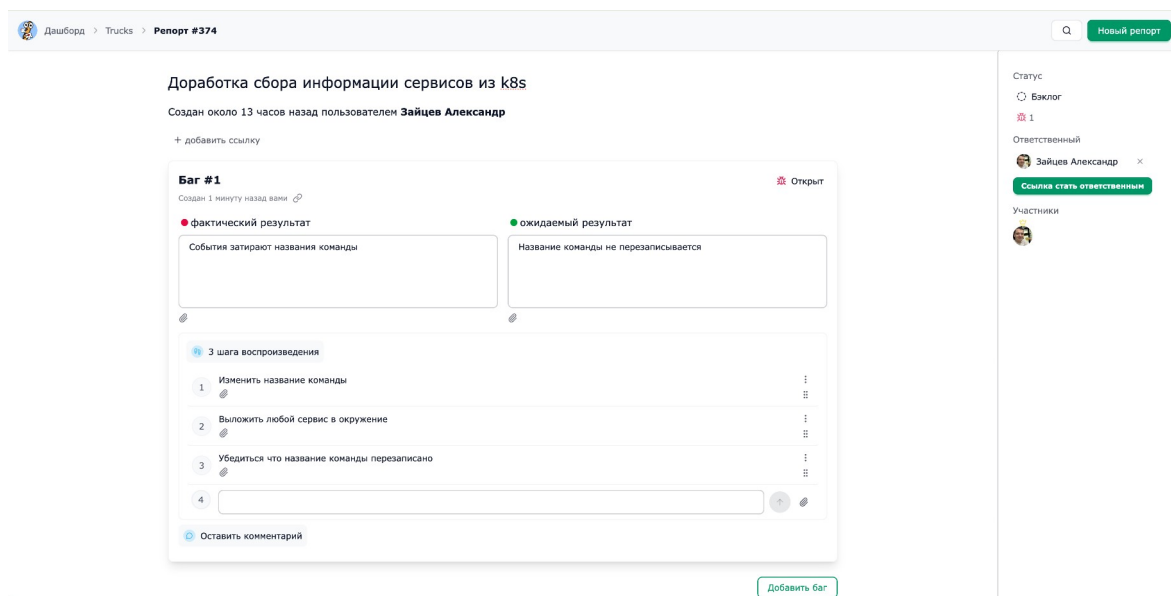


Рисунок 2 — Страница отчета как основной контекст real-time синхронизации

2.2 Исходная реализация real-time взаимодействия

В исходной системе уже присутствует SignalR-хаб `ReportPageHub`, опубликованный по маршруту `/v1/report-page-hub`. Клиентское приложение устанавливает соединение через SignalR и использует автоматическое переподключение. Для подписки на изменения конкретного отчета клиент вызывает метод вступления в группу. Серверная отправка событий централизована в `ReportPageHubClient` [2], [5], [6].

Real-time события охватывают несколько типов изменений: обновление отчета, создание и изменение багов, комментариев, вложений, ссылок и шагов воспроизведения. Это подтверждает, что real-time контур связан с предметной областью системы и используется для совместной работы над сущностями, а не является изолированной демонстрационной функцией.

При этом в исходной инфраструктурной конфигурации `upstream`-блок `app-ari` указывал на один экземпляр серверного сервиса. Даже при наличии Redis в составе общего `compose`-стенда SignalR не использовал Redis как `backplane` для межузловой доставки. Поэтому исходный контур можно охарактеризовать как функциональный одноузловой real-time механизм [3], [4].

2.3 Требования к разрабатываемому решению

Функциональные требования:

1. Система должна поддерживать подключение клиентов к SignalR-хабу через обратный прокси-сервер.
2. Клиент должен иметь возможность вступить в группу, соответствующую отчету.
3. Изменения сущностей отчета должны доставляться всем клиентам, подписанным на группу.
4. Клиенты, подключенные к разным экземплярам серверного сервиса, должны получать одно и то же real-time событие.

5. Клиент должен восстанавливать соединение после разрыва и повторно вступать в необходимые группы.
6. Для демонстрации должен быть доступен диагностический способ определить `serverInstanceId` и `connectionId`.

Нефункциональные требования:

1. Решение должно запускаться на минимальном локальном стенде.
2. Подключение Redis backplane должно быть конфигурируемым через переменную окружения.
3. Реализация должна сохранять совместимость существующего содержимого real-time событий.
4. Доказательная база должна включать воспроизводимые сценарии испытаний.

2.4 Выбор платформы и инструментальных средств

Для реализации масштабируемого real-time контура выбран стек инструментальных средств: ASP.NET Core SignalR для двунаправленного взаимодействия поверх WebSocket с моделью хабов и групп [2], Redis backplane для межузловой доставки событий между экземплярами серверного приложения [4], [9], nginx как обратный прокси-сервер и балансировщик HTTP- и WebSocket-трафика [10], Docker Compose для воспроизводимого запуска режимов с Redis backplane и без него. Проверочный клиент реализован на Node.js и работает напрямую с серверным real-time контуром, поэтому эксперимент не зависит от пользовательского интерфейса продукта.

2.5 Постановка задачи

Необходимо разработать и проверить масштабируемый real-time контур для системы отслеживания задач. Для этого требуется:

1. Добавить возможность подключения SignalR к Redis backplane.
2. Подготовить стенд с двумя экземплярами `app-api`.

3. Настроить nginx так, чтобы входящий трафик распределялся между экземплярами серверного сервиса и корректно проксировались WebSocket-соединения.
4. Добавить диагностические данные, позволяющие установить, к какому узлу подключен клиент.
5. Подготовить простой проверочный клиент, фиксирующий серверный узел, идентификатор соединения и факт получения события.
6. Реализовать автоматизированный сценарий проверки межузловой доставки.
7. Сравнить поведение системы в режимах с Redis backplane и без него.

3. Проектирование масштабируемого real-time контура

3.1 Исходная архитектура

В исходном варианте real-time взаимодействие строится вокруг одного экземпляра `app-api`. Все клиенты устанавливают WebSocket-соединение с одним серверным процессом. Этот процесс хранит сведения о соединениях и группах, а при изменении сущности отправляет событие в нужную группу [3], [5].

Схема исходной архитектуры представлена на рисунке 3.

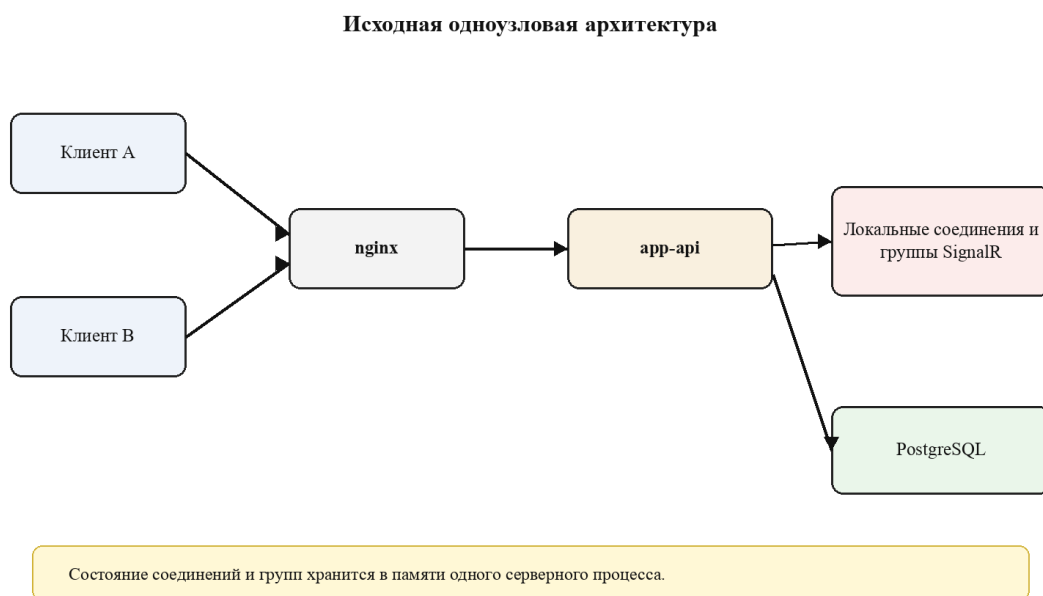


Рисунок 3 — Исходная одноузловая архитектура real-time контура

Недостаток этой архитектуры состоит в том, что она не показывает, как система должна вести себя при нескольких экземплярах `app-api`. Если просто добавить второй экземпляр без `backplane`, каждый процесс будет иметь собственное локальное состояние соединений [3].

Multi-instance вариант без `backplane` представлен на рисунке 4. На нем два экземпляра `app-api` обслуживают собственные наборы WebSocket-соединений и локальные группы `SignalR`. При этом между локальными группами нет канала распространения события, поэтому изменение,

инициированное через первый узел, не становится известным клиентам второго узла.

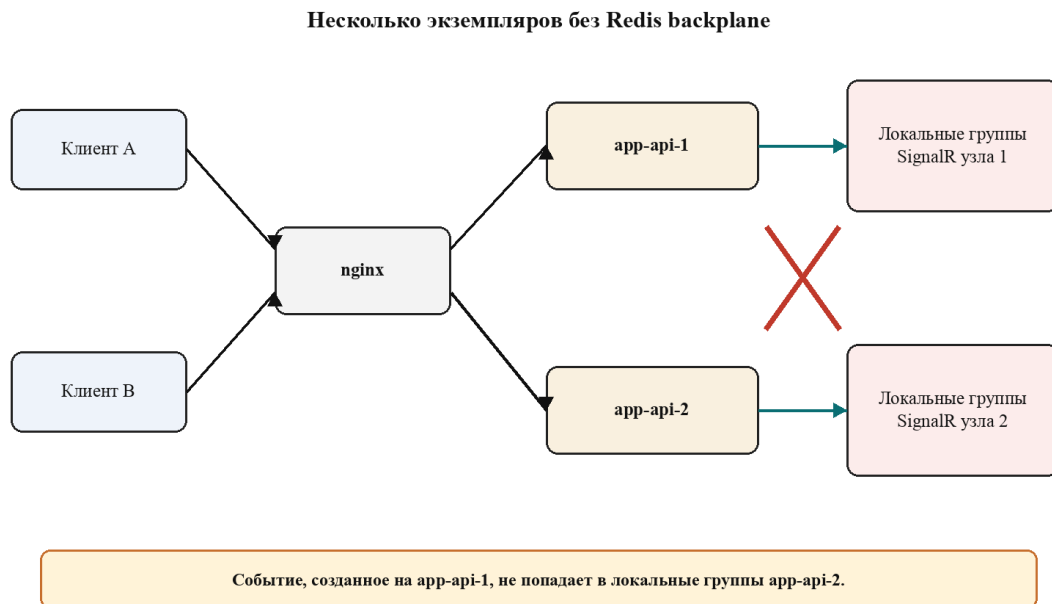


Рисунок 4 — Архитектура с несколькими экземплярами без Redis backplane и фрагментация локальных SignalR-групп

3.2 Целевая архитектура

В целевой архитектуре перед серверными сервисами находится nginx, а приложение запускается в двух экземплярах: app-api-1 и app-api-2. Оба экземпляра подключены к общей базе данных PostgreSQL [17] и к Redis. SignalR использует Redis backplane для межузловой доставки событий [4], [10], [14].

Схема целевой архитектуры представлена на рисунке 5.

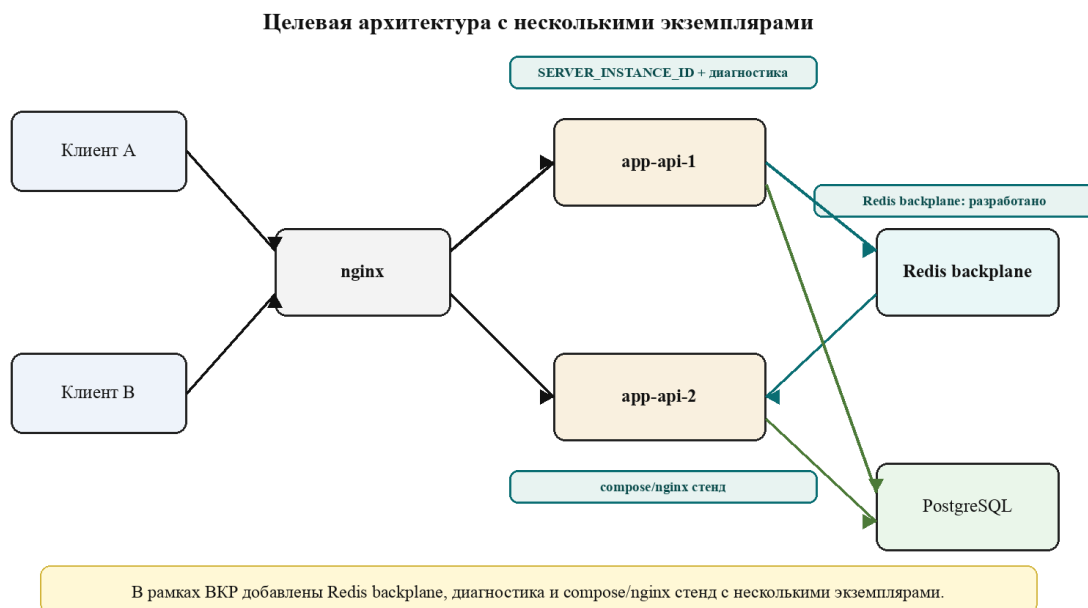


Рисунок 5 — Целевая архитектура с несколькими экземплярами, Redis backplane и элементами, разработанными в рамках ВКР

Целевое свойство архитектуры: если событие создано на `app-api-1`, оно должно быть доставлено клиентам группы, даже если часть этих клиентов подключена к `app-api-2`. Redis backplane выполняет роль механизма межузловой публикации и подписки для SignalR [4], [9].

3.3 Поток real-time события

Поток события в целевой архитектуре выглядит следующим образом:

1. Пользователь выполняет действие в интерфейсе, например изменяет заголовок отчета.
2. HTTP-запрос попадает через nginx на один из экземпляров серверного сервиса.
3. Backend изменяет прикладное состояние в базе данных.
4. `ReportPageHubClient` формирует real-time событие для группы отчета.
5. SignalR отправляет событие локальным клиентам текущего узла.
6. Через Redis backplane событие становится доступным другим экземплярам `app-api`.

7. Другой экземпляр доставляет событие своим клиентам, состоящим в той же группе.
8. Проверочный клиент получает событие и фиксирует результат доставки.

Последовательность межузловой доставки события представлена на рисунке 6.

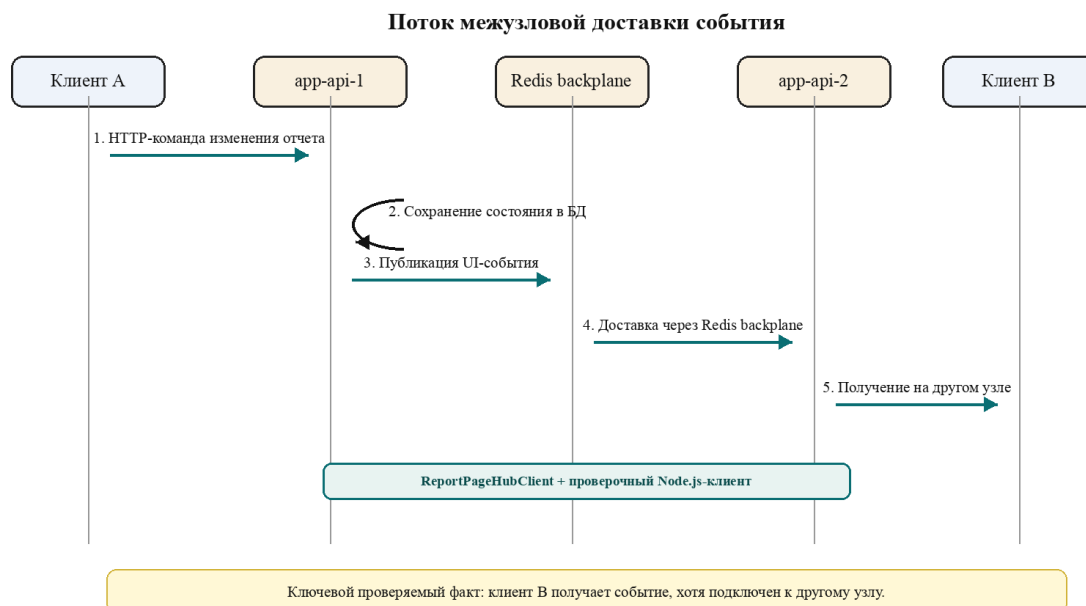


Рисунок 6 — Последовательность межузловой доставки события *ReceiveReportPatch*

3.4 Модель групп и повторной подписки

Группа SignalR соответствует контексту отчета. При открытии страницы отчета клиент вызывает метод вступления в группу. На сервере идентификатор отчета разрешается с учетом *alias/public/team* контекста, после чего формируется ключ группы. Такой подход позволяет объединить в одну *real-time* область всех клиентов, работающих с одним отчетом.

Поскольку *membership* группы является оперативным состоянием SignalR, клиент должен повторять вступление в группу после переподключения. В проверочном клиенте и клиентском *real-time* слое используется автоматическое восстановление соединения и повторное присоединение к группе отчета. Это свойство рассматривается как часть устойчивости *real-time* контура [5], [6].

3.5 Диагностический контур

Для экспериментальной проверки фиксируется не только факт доставки события, но и принадлежность клиентских подключений к разным экземплярам серверного сервиса. Поэтому в проект добавлен диагностический контур:

1. `SERVER_INSTANCE_ID` задает идентификатор экземпляра сервиса.
2. `ServerInstanceInfo` предоставляет идентификатор экземпляра и имя машины.
3. `ReportPageHub.GetConnectionDiagnosticsAsync()` возвращает клиенту `serverInstanceId`, `machineName`, `connectionId` и `userIdentifier`.
4. Backend логирует подключение, вступление в группу, выход из группы, отключение и отправку real-time события.
5. Проверочный клиент выводит диагностические данные в JSON-результате эксперимента.

Такой контур не меняет прикладное содержимое событий, но повышает наблюдаемость стенда [8].

3.6 Семантика доставки и ограничения

Разработанный real-time контур следует рассматривать как механизм интерфейсной синхронизации, а не как систему жесткого промышленного real-time. Redis backplane не делает доставку события мгновенной и синхронно-блокирующей. После изменения состояния событие проходит через локальный экземпляр SignalR, канал Redis и другой экземпляр `app-ari`, поэтому межузловая задержка является ненулевой [4], [9], [14].

В локальном стенде была зафиксирована средняя задержка доставки 7,2 мс и p95 13,8 мс на серии из 30 итераций. Эти значения не являются нагрузочным тестом промышленного уровня, но показывают порядок задержки в контролируемой среде. Для предметной области такая задержка допустима: речь идет об интерфейсной синхронизации отчетов, багов,

комментариев, вложений и статусов, а не о физическом управлении процессом. Источником истины остается PostgreSQL, поэтому при сомнении клиент может повторно запросить актуальное состояние через HTTP.

Ограничение выбранной архитектуры состоит в том, что она не вводит глобальную блокировку нового состояния до подтверждения доставки всеми экземплярами серверного сервиса. Также Redis backplane не обеспечивает долговременную гарантию доставки каждого события интерфейса при полном сбое Redis, клиентского соединения или серверного узла. Если событие потеряно в момент разрыва соединения, восстановление согласованности должно выполняться через повторное вступление в группу и повторное чтение состояния из REST API.

Более строгий промышленный вариант может включать брокер с долговременным хранением событий или журнал событий, версии сущностей, оптимистическую конкурентность, паттерны outbox/inbox, подтверждения потребителей и клиентскую повторную синхронизацию по версии состояния. Такой вариант сильнее с точки зрения гарантий доставки, но существенно усложняет архитектуру и выходит за рамки текущей задачи, где требуется масштабируемая доставка короткоживущих событий интерфейса в существующей SignalR-модели.

4. Реализация решения в целевой системе

4.1 Организация работ

Работы по реализации были разделены на пять направлений: изменение серверной конфигурации SignalR, доработка хаба и централизованного клиента отправки событий, адаптация клиентского real-time слоя к восстановлению соединений, подготовка инфраструктуры стенда с несколькими экземплярами и разработка проверочного клиента для фиксации результатов испытаний. Эти направления охватывают серверную, клиентскую, инфраструктурную и экспериментальную части разработанного решения.

Реализация включает несколько связанных программных артефактов. На стороне серверной части добавлены конфигурируемое подключение Redis backplane, диагностический метод хаба и централизованная отправка событий с логированием `eventId` и `serverInstanceId`. На стороне клиентской части доработан real-time слой страницы отчета: WebSocket-подключение, получение диагностических данных, обработка восстановления соединения и повторное вступление в группу текущего отчета. Отдельно реализован проверочный Node.js-клиент, который воспроизводит межузловую доставку независимо от визуального интерфейса.

4.2 Изменения серверной части

В серверной части добавлена поддержка Redis backplane для SignalR. В конфигурации приложения появилась переменная окружения `REDIS_CONNECTION_STRING`. Если она задана, при инициализации SignalR вызывается подключение `AddStackExchangeRedis`. Если переменная не задана или пустая, приложение продолжает работать в одноузловом режиме без Redis backplane. Это позволяет запускать один и тот же код в двух исследовательских режимах: с межузловой доставкой и без нее [4].

Ключевой фрагмент настройки находится в `backend/bugget-api/Bugget/Extensions/ServiceCollectionExtensions.cs` и приведен в приложении Б. Подключение `backplane` не зашито жестко в код, а управляется конфигурацией окружения. Поэтому тот же исполняемый код можно запускать как в позитивном режиме с `Redis backplane`, так и в негативном контрольном режиме без межузлового канала.

Для разделения каналов `SignalR` используется `prefix app-api-realtime`. Это снижает риск пересечения сообщений с другими возможными приложениями или окружениями, использующими тот же `Redis` [4], [9].

Также добавлена переменная `SERVER_INSTANCE_ID`. Значение используется для логирования и диагностики. Если переменная не задана, сервис может использовать имя машины как `fallback`. На стенде экземпляры получают явные значения `app-api-1` и `app-api-2`.

В `ReportPageHub` добавлен метод `GetConnectionDiagnosticsAsync()`. Он позволяет клиенту получить текущий `connectionId` и идентификатор серверного экземпляра, через который установлено `SignalR`-соединение. Кроме того, в хаб добавлено логирование ключевых событий жизненного цикла соединения: подключение, вступление в группу, выход из группы и отключение [8].

Диагностический метод возвращает данные, необходимые для анализа межузлового сценария: идентификатор экземпляра сервера, имя машины, идентификатор `SignalR`-соединения и пользовательский идентификатор. Фрагмент реализации приведен в приложении Б.

В `ReportPageHubClient` централизована отправка событий в группы через метод `SendToGroupAsync`. При отправке создается `eventId`, логируются имя события, ключ группы, идентификатор серверного экземпляра и исключаемый `connectionId`. Существующее содержимое событий сохранено, что снижает риск регрессий в прикладной логике [5], [8].

Централизация отправки обеспечивает прохождение всех real-time событий отчета через одну точку, где фиксируются `eventId`, группа и экземпляр сервера. Прикладное содержимое событий при этом не меняется: изменяется инфраструктурная семантика доставки и наблюдаемость, а не формат событий для существующего клиентского кода.

4.3 Изменения клиентской части

Клиентская часть решения относится к real-time слою страницы отчета. Он обеспечивает устойчивое SignalR-подключение, хранение диагностических данных и повторную подписку после восстановления соединения. Основные изменения сосредоточены в `frontend/src/shared/model/socket/connection.ts`, `frontend/src/shared/model/socket/model.ts`, `frontend/src/pages/Report/lib/useReportPageSocket.ts` и `frontend/src/pages/Report/lib/reconnectJoinHandler/reconnectJoinHandler.ts`. Соединение строится через JavaScript-клиент SignalR, использует транспорт WebSocket и автоматическое восстановление соединения с нарастающими интервалами ожидания [6].

В приложении В приведены фрагменты клиентского real-time слоя: построение WebSocket-соединения, получение диагностических данных, обработка `onreconnected`, повторное вступление в группу отчета и модульные тесты обработчика восстановления соединения.

После восстановления соединения клиент обновляет `connectionId` и повторно получает диагностические данные. Это обеспечивает корректную передачу `X-Signal-R-Connection-Id` в HTTP-запросах и повышает наблюдаемость стенда.

На странице отчета реализовано повторное вступление в группу после `reconnect`. Текущий идентификатор отчета хранится отдельно, поэтому обработчик восстановления может заново вызвать `JoinReportGroupAsync` для актуальной страницы.

Корректность этого поведения покрыта модульными тестами `reconnectJoinHandler`: проверяются отсутствие вызова без текущего отчета, повторное вступление с текущим идентификатором и использование обновленного идентификатора при последующих `reconnect`.

4.4 Проверочный клиент

Для экспериментальной проверки подготовлен простой Node.js-клиент, который не зависит от пользовательского интерфейса продукта. Клиент напрямую подключается к двум экземплярам `app-api`, вызывает метод `getConnectionDiagnosticsAsync()` и фиксирует `serverInstanceId`, `connectionId`, идентификатор отчета и полученное real-time событие [6], [8].

Такой подход делает эксперимент независимым от полноценного пользовательского интерфейса. Проверяется именно серверный real-time контур и межузловая доставка событий, а не особенности конкретного экрана клиентского приложения.

Клиент использует WebSocket-транспорт SignalR, подключается к двум разным адресам узлов и передает необходимые заголовки тестовой идентичности. При изменении отчета через первый узел клиент ожидает событие на соединении, открытом ко второму узлу.

Фрагмент `scripts/realtime-scaleout-check.mjs` с основной проверяемой последовательностью приведен в приложении В. Проверяемая последовательность включает создание отчета, подключение двух SignalR-соединений к разным узлам, вступление в группу отчета, изменение сущности через первый узел и ожидание события `ReceiveReportPatch` на втором узле.

4.5 Инфраструктурные изменения

Для проверки подготовлен отдельный compose-стенд `docker-compose.thesis.yml`. Он включает:

1. `postgres_app_thesis`;

2. `redis_app_thesis;`
3. `app-api-1;`
4. `app-api-2;`
5. `nginx_app_thesis.`

Оба экземпляра `app-api` собираются из одного `Dockerfile`, подключаются к одной базе данных и одному `Redis`, но получают разные значения `SERVER_INSTANCE_ID`. Для проверки исходного ограничения добавлен `override docker-compose.thesis.no-backplane.yml`, который устанавливает `REDIS_CONNECTION_STRING` в пустую строку для обоих экземпляров.

Отдельный `upstream`-блок `nginx upstreams.thesis.conf` содержит два серверных сервиса [10]:

```
upstream app-api {  
    zone app-api 64k;  
    server app-api-1:7777;  
    server app-api-2:7777;  
}
```

Для стенда добавлена отдельная конфигурация `nginx`, в которой обратный прокси-сервер направляет `HTTP`- и `WebSocket`-трафик на `upstream`-блок `app-api`. Конфигурация не зависит от дополнительных сервисов авторизации или пользовательского профиля и предназначена только для проверки `real-time` контура [10], [14].

4.6 Автоматизированный сценарий проверки

Для воспроизводимой проверки добавлен сценарий `npm run test:realtime-scaleout`. Он выполняет следующие действия:

1. Создает отчет через `app-api-1`.
2. Подключает `SignalR`-клиент А к `app-api-1`.
3. Подключает `SignalR`-клиент В к `app-api-2`.
4. Подписывает оба клиента на группу созданного отчета.
5. Изменяет заголовок отчета через `app-api-1`.
6. Ожидает событие `ReceiveReportPatch` на клиенте В.

7. Измеряет задержку от отправки PATCH-запроса до получения события на клиенте В.

8. Выводит диагностические данные обоих клиентов, полученное содержимое события и метрики доставки.

Такой сценарий исключает случайность балансировки: клиенты намеренно подключаются к разным серверным узлам. Если событие приходит клиенту на втором узле, это является прямым подтверждением межузловой доставки.

Для количественной оценки сценарий поддерживает серийный режим через переменную `THEESIS_ITERATIONS`: скрипт выполняет несколько независимых проверок и выводит число успешных доставок, минимум, максимум, среднее, p50 и p95. Дополнительно предусмотрены режимы `THEESIS_SCENARIO=rejoin` и `THEESIS_SCENARIO=failover` для проверки повторного вступления в группу и переподключения через `nginx` после остановки одного экземпляра `app-api`.

5. Испытания и оценка результатов

5.1 Цель и программа испытаний

Цель испытаний — подтвердить, что реализованное решение устраняет ограничение одноузловой WebSocket-архитектуры и обеспечивает доставку real-time событий между клиентами, подключенными к разным экземплярам серверного сервиса [3], [4].

Программа испытаний включает два основных режима:

1. Режим с несколькими экземплярами без Redis backplane.
2. Режим с несколькими экземплярами и Redis backplane.

Первый режим предназначен для воспроизведения проблемы, второй — для подтверждения результата. Оба режима используют один и тот же код, один и тот же сценарий проверки и отличаются только значением `REDIS_CONNECTION_STRING`. Программа испытаний построена с учётом базовых принципов проверки программного обеспечения [20]: два сопоставимых режима, повторяемость запусков и фиксация результатов в форме, пригодной для последующего анализа.

5.2 Стенд испытаний

Стенд запускается в локальном окружении Docker Compose. В режиме с backplane используются два экземпляра `app-api`, Redis, PostgreSQL и nginx. Проверка выполняется отдельным Node.js-клиентом, который подключается напрямую к real-time контуру и не зависит от дополнительных продуктовых сервисов.

Команда запуска режима с Redis backplane:

```
cd <project-root>
docker compose -f docker-compose.thesis.yml up -d --build
```

Команда запуска режима без Redis backplane:

```
cd <project-root>
docker compose -f docker-compose.thesis.yml -f docker-compose.thesis.no-backplane.yml up -d
--build
```


Проверочный клиент запускается командой:

```
cd <project-root>
node scripts/realtime-scaleout-check.mjs
```

Для серийной проверки доставки и задержки используется команда:

```
cd <project-root>
THEESIS_ITERATIONS=30 node scripts/realtime-scaleout-check.mjs
```

Для проверки восстановления подписки и отказа узла используются команды:

```
THEESIS_SCENARIO=rejoin node scripts/realtime-scaleout-check.mjs
THEESIS_SCENARIO=failover THEESIS_ALLOW_DOCKER_CONTROL=1 THEESIS_TIMEOUT_MS=20000 node
scripts/realtime-scaleout-check.mjs
```

5.3 Результат без Redis backplane

В режиме без Redis backplane оба экземпляра `app-api` работают, но SignalR не имеет межузлового канала доставки событий. Проверочный сценарий завершился истечением времени ожидания:

```
Error: ReceiveReportPatch timed out after 10000ms
```

Интерпретация результата: клиент А был подключен к одному экземпляру сервиса, клиент В — к другому. Событие изменения отчета было создано через первый экземпляр, но не дошло до клиента, подключенного ко второму экземпляру. Этот результат подтверждает исходное архитектурное ограничение: простой запуск нескольких серверных процессов не обеспечивает глобальную real-time доставку [3].

5.4 Результат с Redis backplane

В режиме с Redis backplane проверочный сценарий завершился успешно. Один из зафиксированных результатов:

```
{
  "ok": true,
  "reportId": "3",
  "patchedTitle": "scaleout-patched-1778316874715",
  "nodeA": {
    "serverInstanceId": "app-api-1",
    "machineName": "21ee5de48b13",
    "connectionId": "9iIIs9uWgfJlF6BHXER8XA",
```

```

    "userIdentifier": "thesis-user"
  },
  "nodeB": {
    "serverInstanceId": "app-api-2",
    "machineName": "1d6a8589cdae",
    "connectionId": "NpH7DSAYYJW1EWM2XKhyqQ",
    "userIdentifier": "thesis-user"
  },
  "receivedPatch": {
    "title": "scaleout-patched-1778316874715",
    "updatedAt": "2026-05-09T08:54:35.186614+00:00"
  },
  "metrics": {
    "deliveryLatencyMs": 9.5,
    "patchRequestMs": 8.5
  }
}

```

Из результата видно, что клиент А был подключен к `app-api-1`, а клиент В — к `app-api-2`. При этом клиент В получил событие `ReceiveReportPatch`, инициированное через первый узел. Это подтверждает, что Redis backplane обеспечил межузловую доставку real-time события [4], [9].

5.5 Серийная проверка и задержка доставки

Для дополнительной проверки был выполнен серийный запуск из 30 независимых итераций в режиме с Redis backplane:

```
THEISIS_ITERATIONS=30 node scripts/realtime-scaleout-check.mjs
```

Результат:

```

{
  "ok": true,
  "iterations": 30,
  "successful": 30,
  "failed": 0,
  "latencyMs": {
    "min": 3.8,
    "max": 16.7,
    "avg": 7.2,
    "p50": 6.1,
    "p95": 13.8
  }
}

```

Все 30 проверок завершились успешно: каждый раз клиент, подключенный к `app-api-2`, получал событие, инициированное через `app-api-1`. Измеренная задержка доставки в локальном стенде составила в среднем 7,2 мс, p50 — 6,1 мс, p95 — 13,8 мс. Эти значения не следует трактовать как промышленный нагрузочный тест, поскольку стенд

локальный и сценарий проверяет функциональное свойство доставки, а не предельную пропускную способность. Однако они подтверждают, что реализованная межузловая доставка работает не только в единичном запуске, но и в расширенной серии повторных проверок.

5.6 Проверка восстановления соединения и отказа узла

Режим повторной подписки `THEESIS_SCENARIO=rejoin` проверяет, что после разрыва соединения клиент может создать новое SignalR-подключение, повторно вступить в группу отчета и получить следующее событие. В зафиксированном запуске клиент В сначала был подключен к `app-api-2`, затем получил новое соединение с другим `connectionId`, повторно выполнил `JoinReportGroupAsync` и получил событие `ReceiveReportPatch`. Время повторного подключения и вступления в группу составило 6,5 мс, задержка доставки после повторного вступления — 15,2 мс.

Режим переключения после отказа `THEESIS_SCENARIO=failover` проверяет поведение при остановке одного экземпляра. Клиент В был подключен через `nginx` к `app-api-2`, после чего контейнер `app-api-2` был остановлен. SignalR-клиент переподключился через `nginx` к `app-api-1`, повторно вступил в группу отчета и получил новое событие. Время переподключения и повторного вступления составило 352,0 мс, задержка доставки после переключения — 5,4 мс. Для проверки через балансировщик только по WebSocket в клиенте используется `skipNegotiation`, чтобы установить соединение одним WebSocket-запросом и не зависеть от закрепления сессии на этапе согласования транспорта.

5.7 Сравнительная таблица результатов

Таблица 2 — Сравнительная таблица результатов испытаний

Режим	Backplane	Ожидаемый результат	Фактический результат	Вывод
Несколько экземпляров без Redis backplane	Нет	Событие не доставляется на другой узел	Истечение времени ожидания 10 000 мс	Ограничение воспроизведено

Режим	Backplane	Ожидаемый результат	Фактический результат	Вывод
Несколько экземпляров с Redis backplane	Да	Событие доставлено между узлами	ok: true, получен ReceiveReportPatch	Ограничение устранено
Серия из 30 итераций	Да	Все события доставлены	30 из 30, p95 = 13,8 мс	Подтверждена повторяемость
Повторная подписка после разрыва	Да	Клиент повторно подписывается	Событие после повторной подписки получено, 15,2 мс	Подтверждена повторная подписка
Переключение после отказа узла	Да	Клиент переключается через nginx	app-api-2 → app-api-1, событие получено	Восстановление после отказа

5.8 Проверка входной точки стенда

Дополнительно проверена доступность единой nginx-точки входа. URL `http://localhost:18080/health` возвращает `ok`, а обратный прокси-сервер направляет HTTP- и WebSocket-трафик на upstream-блок `app-api`. Это подтверждает, что демонстрационный стенд пригоден для воспроизводимой проверки real-time контура без зависимости от полноценного пользовательского интерфейса и дополнительных продуктовых сервисов.

5.9 Оценка достоверности результатов

Проведенный эксперимент проверяет ключевое функциональное свойство: доставку события между клиентами, подключенными к разным серверным экземплярам. Достоверность результата обеспечивается тем, что режимы с Redis backplane и без него запускались на одном стенде, с одним и тем же кодом приложения и одним и тем же проверочным сценарием. Единственным существенным отличием между режимами было значение `REDIS_CONNECTION_STRING`.

Сопоставление двух режимов показывает причинную связь между включением межузлового канала SignalR и успешной доставкой события на другой серверный экземпляр. Дополнительные сценарии повторной подписки и переключения после отказа подтверждают, что контур сохраняет работоспособность не только при штатной доставке события, но и после восстановления клиентского подключения.

5.10 Вывод по испытаниям

Испытания подтвердили, что в режиме с несколькими экземплярами без межузловой синхронизации real-time события не доставляются клиентам, подключенным к другому серверному экземпляру. После подключения Redis backplane событие изменения отчета доставляется между `app-api-1` и `app-api-2`. Серийная проверка показала 30 успешных доставок из 30, p95 задержки в локальном стенде составил 13,8 мс. Дополнительно подтверждены повторное вступление в группу после разрыва соединения и восстановление после остановки одного узла. Таким образом, реализованное решение подтверждает заявленный тезис работы: ограничение одноузловой WebSocket-архитектуры устранено за счет распределенного real-time контура [3], [4].

ЗАКЛЮЧЕНИЕ

В ходе работы была рассмотрена проблема построения real-time интерфейса для системы отслеживания задач в условиях горизонтального масштабирования WebSocket-соединений. Показано, что простая одноузловая реализация SignalR не обеспечивает корректную работу при нескольких экземплярах серверного сервиса, поскольку сведения о подключениях и группах локальны для каждого процесса [3], [5].

Для решения проблемы была выбрана архитектура на базе нескольких экземпляров `app-api`, `nginx` как обратного прокси-сервера и Redis backplane для межузловой доставки SignalR-событий. В целевой системе реализована поддержка конфигурируемого Redis backplane, добавлены идентификаторы серверных экземпляров, диагностический метод хаба, логирование real-time событий, проверочный клиент и воспроизводимый Docker Compose стенд [4], [8], [10], [14].

Экспериментальная проверка показала различие между двумя режимами. При отключенном Redis backplane событие не доставлялось клиенту, подключенному к другому серверному экземпляру. При

включенном Redis backplane тот же сценарий завершился успешно: клиент на `app-api-2` получил событие, инициированное через `app-api-1`.

Серийный запуск из 30 итераций подтвердил повторяемость результата и позволил получить первичную оценку задержки доставки. Сценарии повторной подписки и переключения после отказа показали, что клиент может восстановить рабочую подписку после разрыва соединения и после остановки одного серверного экземпляра. Это подтверждает устранение ограничения одноузловой WebSocket-архитектуры [3], [4].

Полученные результаты имеют практическую значимость для дальнейшего развития системы отслеживания задач, поскольку предложенное решение позволяет перейти от локального real-time механизма к масштабируемому контуру совместной работы над сущностями. Кроме масштабирования, решение повышает отказоустойчивость приложения: при наличии нескольких серверных экземпляров и повторного вступления клиента в группы система может продолжить доставку событий после потери одного узла, тогда как одноузловая архитектура такой возможности не предоставляет.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Fette I., Melnikov A. The WebSocket Protocol: RFC 6455. IETF, 2011. URL: <https://www.rfc-editor.org/rfc/rfc6455> (дата обращения: 09.05.2026).
2. Microsoft. Overview of ASP.NET Core SignalR. URL: <https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction> (дата обращения: 09.05.2026).
3. Microsoft. ASP.NET Core SignalR production hosting and scaling. URL: <https://learn.microsoft.com/en-us/aspnet/core/signalr/scale> (дата обращения: 09.05.2026).
4. Microsoft. Redis backplane for ASP.NET Core SignalR scale-out. URL: <https://learn.microsoft.com/en-us/aspnet/core/signalr/redis-backplane> (дата обращения: 09.05.2026).
5. Microsoft. Manage users and groups in SignalR. URL: <https://learn.microsoft.com/en-us/aspnet/core/signalr/groups> (дата обращения: 09.05.2026).
6. Microsoft. ASP.NET Core SignalR JavaScript client. URL: <https://learn.microsoft.com/en-us/aspnet/core/signalr/javascript-client> (дата обращения: 09.05.2026).
7. Microsoft. ASP.NET Core SignalR configuration. URL: <https://learn.microsoft.com/en-us/aspnet/core/signalr/configuration> (дата обращения: 09.05.2026).
8. Microsoft. Logging and diagnostics in ASP.NET Core SignalR. URL: <https://learn.microsoft.com/en-us/aspnet/core/signalr/diagnostics> (дата обращения: 09.05.2026).
9. Redis Ltd. Redis Pub/Sub. URL: <https://redis.io/docs/latest/develop/pubsub/> (дата обращения: 09.05.2026).
10. NGINX. WebSocket proxying. URL: <https://nginx.org/en/docs/http/websocket.html> (дата обращения: 09.05.2026).
11. ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. М.: Стандартинформ, 2018.
12. ГОСТ Р 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. М.: Стандартинформ, 2018.
13. Рогачев С. А., Бабюк Ю. Программная инженерия. Требования к подготовке и содержанию отчетов по практике: учебно-методическое пособие. СПб.: ГУАП, 2025. 40 с.
14. Олифер В. Г., Олифер Н. А. Компьютерные сети. Принципы, технологии, протоколы: учебник. 6-е изд. СПб.: Питер, 2022. 1008 с.
15. Орлов С. А. Программная инженерия: учебник для вузов. 5-е изд. СПб.: Питер, 2016. 640 с.
16. Гагарина Л. Г., Кокорева Е. В., Виснадул Б. Д. Технология разработки программного обеспечения: учебное пособие. М.: ФОРУМ: ИНФРА-М, 2021. 400 с.
17. Советов Б. Я., Цехановский В. В., Чертовской В. Д. Базы данных: теория и практика: учебник. М.: Юрайт, 2020. 463 с.
18. Гвоздева В. А., Баллод Б. А. Основы построения автоматизированных информационных систем: учебник. М.: ФОРУМ: ИНФРА-М, 2020. 320 с.
19. Липаев В. В. Программная инженерия. Методологические основы: учебник. М.: ТЕИС, 2006. 608 с.
20. Куликов С. С. Тестирование программного обеспечения. Базовый курс. Минск: Четыре четверти, 2017. 312 с.

ПРИЛОЖЕНИЕ А

Конфигурация стенда

Фрагмент `docker-compose.thesis.yml` задает два экземпляра серверного сервиса, общий Redis и диагностические идентификаторы серверных узлов.

```
app-api-1:
  build:
    context: ./backend/bugget-api
    dockerfile: Bugget/Dockerfile
  environment:
    POSTGRES_CONNECTION_STRING: Host=db;Port=5432;Database=app_db;Username=postgres
    REDIS_CONNECTION_STRING: redis:6379
    SERVER_INSTANCE_ID: app-api-1
  ports: ["7771:7777"]

app-api-2:
  build:
    context: ./backend/bugget-api
    dockerfile: Bugget/Dockerfile
  environment:
    POSTGRES_CONNECTION_STRING: Host=db;Port=5432;Database=app_db;Username=postgres
    REDIS_CONNECTION_STRING: redis:6379
    SERVER_INSTANCE_ID: app-api-2
  ports: ["7772:7777"]

redis:
  image: redis:8
  container_name: redis_app_thesis
```

Фрагмент `docker-compose.thesis.no-backplane.yml` используется для воспроизведения исходного ограничения режима с несколькими экземплярами.

```
services:
  app-api-1:
    environment:
      REDIS_CONNECTION_STRING: ""

  app-api-2:
    environment:
      REDIS_CONNECTION_STRING:
```

Фрагмент `upstreams.thesis.conf` задает upstream-блок nginx для балансировки HTTP- и WebSocket-трафика между двумя экземплярами `app-api`.

```
upstream app-api {
    zone app-api 64k;
    server app-api-1:7777;
    server app-api-2:7777;
}
```


ПРИЛОЖЕНИЕ Б

Фрагменты реализации серверной части

Настройка SignalR-конвейера выполнена в методе `AddMessaging` расширения `ServiceCollectionExtensions`. Сначала создается базовая регистрация хаба и JSON-протокола, затем условно подключается Redis backplane: при наличии переменной окружения `REDIS_CONNECTION_STRING` тот же исполняемый код переключается в режим межузловой доставки, при пустом значении — продолжает работать в одноузловом режиме. Это позволяет запускать один и тот же образ в позитивном и негативном контрольных режимах эксперимента.

```
public static IServiceCollection AddMessaging(this IServiceCollection services)
{
    services.AddSingleton<IUserIdProvider, SignalRUserIdProvider>();

    var signalRBuilder = services.AddSignalR(options =>
    {
        options.EnableDetailedErrors = true;
        options.KeepAliveInterval = TimeSpan.FromSeconds(15);
        options.ClientTimeoutInterval = TimeSpan.FromSeconds(60);
    })
    .AddJsonProtocol(options =>
    {
        options.PayloadSerializerOptions.DefaultIgnoreCondition =
            JsonIgnoreCondition.WhenWritingNull;
    })
    .AddHubOptions<ReportPageHub>(options =>
    {
        options.AddFilter<HubExceptionHandlerFilter>();
    });

    var redisConnectionString =
        Environment.GetEnvironmentVariable(Environment.Constants.RedisConnectionString);
    if (!string.IsNullOrEmpty(redisConnectionString))
    {
        signalRBuilder.AddStackExchangeRedis(redisConnectionString, options =>
        {
            options.Configuration.ChannelPrefix =
                RedisChannel.Literal("app-api-realtime");
        });
    }

    return services;
}

// Регистрация централизованного отправителя событий в группу
services.AddSingleton<IReportPageHubClient, ReportPageHubClient>();
// Информация об экземпляре сервера, инжектируется в хаб и в отправитель событий
services.AddSingleton<ServerInstanceInfo>();
```

Контракт диагностических данных оформлен как `record` и используется в качестве возвращаемого типа метода хаба `GetConnectionDiagnosticsAsync`. В контракт включены

идентификатор экземпляра сервера, имя машины, идентификатор SignalR-соединения и пользовательский идентификатор.

```
public sealed record RealtimeConnectionDiagnostics(  
    string ServerInstanceId,  
    string MachineName,  
    string ConnectionId,  
    string? UserIdentifier);
```

Диагностический метод хаба возвращает идентификатор серверного экземпляра и соединения, что позволяет доказать подключение клиентов к разным узлам и сопоставить серверные логи с действиями конкретного клиента.

```
public Task<RealtimeConnectionDiagnostics> GetConnectionDiagnosticsAsync()  
{  
    return Task.FromResult(new RealtimeConnectionDiagnostics(  
        serverInstanceInfo.Id,  
        serverInstanceInfo.MachineName,  
        Context.ConnectionId,  
        Context.UserIdentifier  
    ));  
}
```

Жизненный цикл соединения сопровождается журналированием с указанием идентификатора экземпляра сервера. Это позволяет в негативном и позитивном сценариях соотнести события подключения и отключения клиента с конкретным узлом.

```
public override async Task OnConnectedAsync()  
{  
    logger.LogWarning(  
        "Клиент {@ConnectionId} подключился к SignalR на экземпляре {@ServerInstanceId}",  
        Context.ConnectionId,  
        serverInstanceInfo.Id);  
  
    await base.OnConnectedAsync();  
}  
  
public override async Task OnDisconnectedAsync(Exception? exception)  
{  
    logger.LogWarning(  
        "Клиент {@ConnectionId} отключился от экземпляра {@ServerInstanceId}. Причина:  
        {@Reason}",  
        Context.ConnectionId,  
        serverInstanceInfo.Id,  
        exception?.Message ?? "неизвестно");  
  
    await base.OnDisconnectedAsync(exception);  
}
```

Подписка клиента на real-time события отчета реализована как доменная операция, а не как прямой вызов `Groups.AddToGroupAsync`.

Метод проверяет авторизацию пользователя, разрешает идентификатор отчета по нескольким источникам (внутренний, публичный и командный) и формирует ключ группы через структуру ReportIdContext. Только после доменной проверки выполняется присоединение соединения к группе SignalR.

```
public async Task JoinReportGroupAsync(string aliasId)
{
    var user = Context.User?.GetIdentity();
    if (user is null)
    {
        throw new HubException("пользователь не авторизован");
    }

    var (reportId, publicId, teamReportId) =
        ReportIdResolveHelper.ResolveReportId(aliasId, aliasOptions.Value);
    var resolvedReport = await reportsService.ResolveReportIdAsync(
        user.OrganizationId,
        user.TeamId,
        reportId,
        publicId,
        teamReportId);

    if (resolvedReport == null)
    {
        throw new HubException("репорт не найден");
    }

    var groupKey = new ReportIdContext(
        resolvedReport.Id,
        aliasId,
        resolvedReport.CreatorTeamId
    ).GroupKey;

    logger.LogWarning(
        "Клиент {@ConnectionId} подключился к группе {@ReportId} на экземпляре  

        {@ServerInstanceId}",
        Context.ConnectionId,
        groupKey,
        serverInstanceInfo.Id);

    await Groups.AddToGroupAsync(Context.ConnectionId, groupKey);
}
```

Централизованная отправка события в группу сохраняет существующее содержимое событий и добавляет диагностическое логирование. Поддерживается режим исключения соединения-инициатора через GroupExcept, что позволяет не дублировать клиенту-инициатору событие, на которое он уже среагировал по HTTP-ответу.

```
private Task SendToGroupAsync(
    string groupKey,
    string eventName,
    object?[] args,
    string? excludedConnectionId = null)
{
    var eventId = Guid.NewGuid().ToString("N");

    logger.LogInformation(
```

```

        "Realtime event {@EventId} {@EventName} отправлен из {@ServerInstanceId} в группу
        {@GroupKey}, excludedConnectionId={@ExcludedConnectionId}",
        eventId,
        eventName,
        serverInstanceInfo.Id,
        groupKey,
        excludedConnectionId);

    var clients = excludedConnectionId is null
        ? hubContext.Clients.Group(groupKey)
        : hubContext.Clients.GroupExcept(groupKey, excludedConnectionId);

    return clients.SendCoreAsync(eventName, args);
}

```

На основе единого метода `SendToGroupAsync` построен набор типизированных доменных отправителей. Real-time контур покрывает не одно событие, а полный набор сущностей страницы отчета: сам отчет, баги, шаги воспроизведения, комментарии, вложения и ссылки. Ниже приведена выжимка из 17 методов класса `ReportPageHubClient`, показывающая разнообразие транслируемых событий и переключение имени события по типу вложения.

```

public Task SendReportPatchAsync(
    string groupKey, PatchReportSocketView view, string? signalRConnectionId)
    => SendToGroupAsync(groupKey, "ReceiveReportPatch", [view], signalRConnectionId);

public Task SendBugCreateAsync(
    string groupKey, BugSummaryDbModel summary, string? signalRConnectionId)
    => SendToGroupAsync(groupKey, "ReceiveBugCreate", [summary], signalRConnectionId);

public Task SendCommentCreateAsync(
    string groupKey, CommentSummaryDbModel comment, string? signalRConnectionId)
    => SendToGroupAsync(groupKey, "ReceiveCommentCreate", [comment], signalRConnectionId);

public Task SendAttachmentCreateAsync(
    string groupKey, AttachmentSocketView view, string? signalRConnectionId)
{
    string eventName = view.AttachType switch
    {
        (int)AttachType.Comment => "ReceiveCommentAttachmentCreate",
        (int)AttachType.BugStep => "ReceiveBugStepAttachmentCreate",
        _ => "ReceiveBugAttachmentCreate"
    };

    return SendToGroupAsync(groupKey, eventName, [view], signalRConnectionId);
}

public Task SendBugStepsOrderUpdateAsync(
    string groupKey, int bugId, BugStepSummaryDbModel[] steps,
    string? signalRConnectionId)
    => SendToGroupAsync(
        groupKey, "ReceiveBugStepsOrderUpdate", [bugId, steps], signalRConnectionId);

```

ПРИЛОЖЕНИЕ В

Фрагменты клиентского real-time слоя, проверочный клиент и результаты испытаний

Клиентская часть решения относится к real-time слою страницы отчета: построению SignalR WebSocket-соединения, типизированному контракту событий, регистрации единого набора обработчиков, обновлению диагностических данных, повторному вступлению в группу после восстановления соединения, доменной обработке событий и тестам этого поведения.

```
const reconnectDelays = [0, 2_000, 5_000, 10_000, 30_000, 60_000, 120_000];

export const buildConnection = (): HubConnection => {
  const wsPath = getAppWebSocketUrl("/report-page-hub", "v1");
  const fullUrl = `${window.location.origin}${wsPath}`;

  return new HubConnectionBuilder()
    .withUrl(fullUrl, {
      transport: HttpTransportType.WebSockets,
      skipNegotiation: import.meta.env.VITE_SIGNALR_SKIP_NEGOTIATION === "true",
    })
    .withAutomaticReconnect(reconnectDelays)
    .build();
};
```

Контракт real-time событий типизирован: серверные имена событий перечислены в enum `SocketEvent`, а для каждого события задан тип полезной нагрузки в `SocketPayload`. Для событий, аргументы которых на стороне сервера передаются позиционно, заданы кастомные парсеры, преобразующие их в объект с именованными полями.

```
export enum SocketEvent {
  ReportPatch = "ReceiveReportPatch",
  ReportParticipant = "ReceiveReportParticipant",
  ReportLinkCreate = "ReceiveReportLinkCreate",
  ReportLinkUpdate = "ReceiveReportLinkUpdate",
  ReportLinkDelete = "ReceiveReportLinkDelete",
  BugCreate = "ReceiveBugCreate",
  BugPatch = "ReceiveBugPatch",
  CommentCreate = "ReceiveCommentCreate",
  CommentUpdate = "ReceiveCommentUpdate",
  CommentDelete = "ReceiveCommentDelete",
  BugStepCreate = "ReceiveBugStepCreate",
  BugStepPatch = "ReceiveBugStepPatch",
  BugStepsOrderUpdate = "ReceiveBugStepsOrderUpdate",
  BugStepDelete = "ReceiveBugStepDelete",
  BugAttachmentCreate = "ReceiveBugAttachmentCreate",
  // ... всего 24 события для страницы отчета
}

export type SocketPayload = {
  [SocketEvent.ReportPatch]: PatchReportSocketResponse;
  [SocketEvent.BugPatch]: { bugId: number; patch: PatchBugSocketResponse };
  [SocketEvent.CommentDelete]: { bugId: number; commentId: number };
};
```

```

// ... соответствующие типы для остальных событий
};

export const customParsers: Partial<
  Record<SocketEvent, (...args: unknown[]) => SocketPayload[SocketEvent]>
> = {
  [SocketEvent.BugPatch]: (...args) => {
    const [bugId, patch] = args as [number, PatchBugSocketResponse];
    return { bugId, patch };
  },
  [SocketEvent.CommentDelete]: (...args) => {
    const [bugId, commentId] = args as [number, number];
    return { bugId, commentId };
  },
};

```

Команды на серверный хаб оформлены как effector-эффекты. Это делает работу с SignalR частью реактивной модели приложения и позволяет наблюдать за состоянием подписки тем же способом, что и за остальными доменными операциями.

```

export const joinReportFx = socket.createEffect(
  async ({ conn, reportId }: { conn: HubConnection; reportId: string }) => {
    await conn.invoke("JoinReportGroupAsync", reportId);
  }
);

export const leaveReportFx = socket.createEffect(
  async ({ conn, reportId }: { conn: HubConnection; reportId: string }) => {
    await conn.invoke("LeaveReportGroupAsync", reportId);
  }
);

```

Инициализация соединения построена как единая последовательность: создается `HubConnection`, по перечислению `SocketEvent` регистрируется единый набор обработчиков с поддержкой кастомных парсеров, подписки на системные события `onreconnected` и `onclose` обеспечивают восстановление и корректную очистку. Защита через `initPromise` исключает повторную параллельную инициализацию.

```

export const initSocketFx = socket.createEffect(async () => {
  const currentConn = $connection.getState();
  if (currentConn && currentConn.state !== HubConnectionState.Disconnected) {
    return;
  }

  if (initPromise) {
    await initPromise;
    return;
  }

  initPromise = (async () => {
    const conn = buildConnection();
    const handlers = new Map<SocketEvent, (p: unknown) => void>();

    Object.values(SocketEvent).forEach((event) => {
      const customParser = customParsers[event];

```

```

const handler = (...args: unknown[]) => {
  const payload = customParser
    ? customParser(...args)
    : (args[0] as SocketPayload[SocketEvent]);
  socketEventReceived({ type: event, payload });
};
conn.on(event, handler);
handlers.set(event, handler);
});

conn.onreconnected((connectionId) => {
  connectionReconnected();
  setSignalRConnectionId(connectionId ?? null);
  void refreshConnectionDiagnostics(conn);
});

conn.onclose((e) => {
  handlers.forEach((h, ev) => conn.off(ev, h));
  if ($connection.getState() === (conn as ConnectionReady)) {
    connectionClosed(e);
    setSignalRConnectionId(null);
  }
});

try {
  await startConnection(conn);
  connectionStarted(
    Object.assign(conn, { started: true }) as ConnectionReady
  );
  await refreshConnectionDiagnostics(conn);
} catch (e) {
  handlers.forEach((h, ev) => conn.off(ev, h));
  connectionClosed(e as Error);
}
})();

try { await initPromise; } finally { initPromise = null; }
});

```

После восстановления соединения клиент обновляет диагностические данные и сохраняет актуальный `connectionId`; это позволяет исключать собственное соединение при отправке события и проверять, к какому узлу подключен клиент после `reconnect`.

```

const refreshConnectionDiagnostics = async (conn: HubConnection) => {
  try {
    const diagnostics = await conn.invoke<ConnectionDiagnostics>(
      "GetConnectionDiagnosticsAsync"
    );
    setSignalRConnectionId(diagnostics.connectionId);
    connectionDiagnosticsUpdated(diagnostics);
  } catch (e) {
    console.error("[Socket] Failed to read connection diagnostics", e);
    connectionDiagnosticsUpdated(null);
  }
};

```

Страница отчета хранит текущий `reportId` и после `reconnect` повторно вызывает `JoinReportGroupAsync`, чтобы восстановить `membership` в SignalR-группе. Логика повторной подписки выделена в

отдельный модуль `createReconnectJoinHandler`, что упрощает тестирование.

```
const currentReportId = useRef<string | null>(null);

useEffect(() => {
  if (!connection) return;

  connection.onreconnected(
    createReconnectJoinHandler({
      join,
      getCurrentReportId: () => currentReportId.current,
    })
  );
}, [connection, join]);

export const createReconnectJoinHandler = ({ join, getCurrentReportId }) => {
  return () => {
    const reportId = getCurrentReportId();
    if (reportId != null) join(reportId);
  };
};
```

Поведение повторного вступления в группу покрыто модульными тестами: проверяются отсутствие лишнего вызова без текущего отчета, повторная подписка после ресонnect и использование актуального `reportId` при последовательных переподключениях.

```
it("does not call join when there is no current report id", () => {
  const calls: string[] = [];
  const handler = createReconnectJoinHandler({
    join: (reportId) => calls.push(reportId),
    getCurrentReportId: () => null,
  });

  handler();

  expect(calls).toEqual([]);
});

it("calls join with current report id after reconnect", () => {
  const calls: string[] = [];
  const handler = createReconnectJoinHandler({
    join: (reportId) => calls.push(reportId),
    getCurrentReportId: () => "42",
  });

  handler();

  expect(calls).toEqual(["42"]);
});

it("uses latest report id on each reconnect", () => {
  const calls: string[] = [];
  let currentReportId = "101";
  const handler = createReconnectJoinHandler({
    join: (reportId) => calls.push(reportId),
    getCurrentReportId: () => currentReportId,
  });

  handler();
  currentReportId = "202";
  handler();
});
```



```
expect(calls).toEqual(["101", "202"]);
});
```

Полученные real-time события распределяются по доменным сторам страницы отчета: изменение отчета, создание и изменение бага, операции над комментариями, шагами воспроизведения и вложениями. Ниже приведена выжимка из 24 подписок хука useReportSocketEvents, демонстрирующая разложение real-time потока на доменные операции интерфейса.

```
export const useReportSocketEvents = () => {
  const reportId = useUnit($reportIdStore);
  const socketEvents = useUnit({
    patchReportSocketEvent,
    createBugSocketEvent,
    patchBugSocketEvent,
    createCommentSocketEvent,
    updateCommentSocketEvent,
    deleteCommentSocketEvent,
    bugAttachmentCreatedSocketEvent,
    // ...
  });

  useSocketEvent(SocketEvent.ReportPatch, (patch) =>
    socketEvents.patchReportSocketEvent(patch)
  );

  useSocketEvent(SocketEvent.BugCreate, (bug) => {
    if (!reportId) return;
    socketEvents.createBugSocketEvent({ reportId, bug });
  });

  useSocketEvent(SocketEvent.BugPatch, ({ bugId, patch }) =>
    socketEvents.patchBugSocketEvent({ bugId, patch })
  );

  useSocketEvent(SocketEvent.CommentCreate, (comment) =>
    socketEvents.createCommentSocketEvent(comment)
  );

  useSocketEvent(SocketEvent.CommentUpdate, (comment) =>
    socketEvents.updateCommentSocketEvent(comment)
  );

  useSocketEvent(SocketEvent.CommentDelete, ({ bugId, commentId }) =>
    socketEvents.deleteCommentSocketEvent({ bugId, commentId })
  );

  useSocketEvent(SocketEvent.BugAttachmentCreate, (attachment) => {
    if (attachment.attachType === AttachmentTypes.COMMENT) return;
    socketEvents.bugAttachmentCreatedSocketEvent(attachment);
  });
  // ... остальные подписки по аналогии
};
```

Проверочный клиент

Чтобы изоляция от пользовательского интерфейса не мешала строгости проверки, проверочный клиент имеет собственные обязанности: единый таймаут ожидания событий, доменный HTTP-запрос на изменение отчета с пробросом `X-Signal-R-Connection-Id` (для исключения инициатора в `GroupExcept`), а также привязка соединения к конкретному серверному экземпляру по идентификатору `serverInstanceId`. Последнее особенно важно через `nginx`, который сам распределяет соединение между узлами.

```
const withTimeout = (promise, label) => {
  let timer;
  const timeout = new Promise( (_, reject) => {
    timer = setTimeout(
      () => reject(new Error(`${label} timed out after ${timeoutMs}ms`)),
      timeoutMs,
    );
  });

  return Promise.race([promise, timeout]).finally(() => clearTimeout(timer));
};

const sendReportPatch = async (reportId, title, signalRConnectionId) =>
  requestJson(`${nodeA}/v2/reports/${reportId}`, {
    method: "PATCH",
    headers: { "X-Signal-R-Connection-Id": signalRConnectionId },
    body: JSON.stringify({ title }),
  });

const connectToServerInstance = async (baseUrl, serverInstanceId) => {
  for (let attempt = 1; attempt <= 8; attempt += 1) {
    const connection = buildConnection(baseUrl, { automaticReconnect: true });
    await connection.start();
    const diagnostics = await getDiagnostics(connection);

    if (diagnostics.serverInstanceId === serverInstanceId) {
      return { connection, diagnostics, attempt };
    }

    await connection.stop();
    await wait(150);
  }

  throw new Error(
    `Could not connect to ${serverInstanceId} through ${baseUrl}`,
  );
};
```

Базовый сценарий проверки доставки события подключает два SignalR-соединения к разным узлам, подписывает их на группу отчета и ожидает событие `ReceiveReportPatch` на втором узле. Замеры задержки выполняются по `performance.now()` между моментом отправки HTTP-запроса и моментом прихода события на втором клиенте.

```
const diagnosticsA = await getDiagnostics(clientA);
```

```

const diagnosticsB = await getDiagnostics(clientB);

await clientA.invoke("JoinReportGroupAsync", reportId);
await clientB.invoke("JoinReportGroupAsync", reportId);

const patchStartedAt = performance.now();
await sendReportPatch(reportId, patchedTitle, diagnosticsA.connectionId);
await withTimeout(patchWatcher.promise, "ReceiveReportPatch");

return {
  ok: patchWatcher.getReceivedPatch()?.title === patchedTitle,
  nodeA: diagnosticsA,
  nodeB: diagnosticsB,
  metrics: {
    deliveryLatencyMs: roundMetric(patchWatcher.getReceivedAt() - patchStartedAt),
  },
};

```

Сценарий проверки отказоустойчивости останавливает контейнер целевого узла и ожидает автоматический переход клиента на оставшийся узел через nginx. Подписывается обработчик `onreconnected`, который после восстановления соединения читает диагностику нового узла и повторно вызывает `JoinReportGroupAsync`. Сценарий считается пройденным, если событие изменения отчета доставлено клиенту на новом узле, а `serverInstanceId` после восстановления отличается от исходного.

```

const initial = await connectToServerInstance(nginxNode, failoverTargetInstance);
clientB = initial.connection;
const diagnosticsB = initial.diagnostics;

await clientB.invoke("JoinReportGroupAsync", reportId);

const reconnectStartedAt = performance.now();
const rejoinPromise = new Promise((resolve, reject) => {
  clientB.onreconnected(async () => {
    try {
      diagnosticsAfterFailover = await getDiagnostics(clientB);
      await clientB.invoke("JoinReportGroupAsync", reportId);
      resolve();
    } catch (error) {
      reject(error);
    }
  });
});

await runDocker("stop", "--time", "1", failoverContainer);
await withTimeout(rejoinPromise, "SignalR failover reconnect/rejoin");
const rejoinedAt = performance.now();

const secondPatchWatcher = waitForReportPatch(clientB, secondTitle);
await sendReportPatch(reportId, secondTitle, diagnosticsA.connectionId);
await withTimeout(secondPatchWatcher.promise, "Post-failover ReceiveReportPatch");

return {
  ok:
    secondPatchWatcher.getReceivedPatch()?.title === secondTitle &&
    diagnosticsAfterFailover?.serverInstanceId !== diagnosticsB.serverInstanceId,
  metrics: {
    failoverReconnectAndRejoinMs: roundMetric(rejoinedAt - reconnectStartedAt),
  },
};

```

Результаты испытаний фиксируют различие между режимом без backplane и режимом с Redis backplane.

```
Multi-instance без backplane:  
  Error: ReceiveReportPatch timed out after 10000ms
```

```
Multi-instance с Redis backplane:  
  ok: true  
  nodeA.serverInstanceId: app-api-1  
  nodeB.serverInstanceId: app-api-2  
  deliveryLatencyMs: 9.5
```

```
THESIS_ITERATIONS=30:  
  successful: 30  
  failed: 0  
  avg: 7.2 ms  
  p50: 6.1 ms  
  p95: 13.8 ms
```

```
THESIS_SCENARIO=rejoin:  
  reconnectAndRejoinMs: 6.5  
  deliveryLatencyMs: 15.2
```

```
THESIS_SCENARIO=failover:  
  app-api-2 -> app-api-1  
  failoverReconnectAndRejoinMs: 352.0  
  deliveryLatencyMs: 5.4
```